



B Automation Services



NextState
AUTOMATION

RoboSync Hub Custom KRL Software

Client: Statement Fabrication

Author	NextState Automation Ltd (helpdesk@nextstateautomation.co.uk)
Software version	0.1.1
Manual revision	Issue 1.0
Target	KUKA KR C4 / KSS 8.x controller, with a Dyze RoboSync Hub connected through an SST gateway
Reference documents	RoboSync Hub User Guide; RoboSync Hub Gateway Mapping
Status	Not yet validated on the robot.



1 Contents

1	Introduction.....	3
1.1	Representation of warning and notes.....	3
1.2	Relation of other documentation.....	4
1.3	Files.....	4
1.4	Architecture.....	5
2	Function reference.....	5
2.1	RoboSyncPidControl.....	5
2.2	RoboSyncStepper.....	7
2.2.1	Extrusion speed calculation.....	8
3	Background Logic – RoboSyncSps.....	8
3.1	Runs Per cycle.....	8
3.2	Stepper State Machine.....	8
3.2.1	Heater Idle timeout.....	11
3.2.2	Resume interrupt RoboSyncInterrupt.....	11
3.3	Operator Messages.....	11
3.4	Configuration Variables.....	12
4	Using RoboSync module In a program.....	13
4.1	Example program.....	13
4.2	Switching extrusion on the path.....	14



1 Introduction

RoboSync lets a KUKA robot program run a Dyze RoboSync Hub directly from KRL. It controls three things:

- **Up to four PID temperature controllers** in the hub: 0 heated bed, 1 top heater, 2 bottom heater, 3 nozzle heater. You can set and read the setpoint, the setpoint limits, the alarm limit, the PID gains and autotune.
- **The extruder stepper drive.** It runs in jog mode, and its speed follows the robot's TCP speed so the extruded volume matches the bead being laid.
- **Safety and recovery:** the program stops on drive or heater alarms, alarms are shown as operator messages, the extruder stops when the program stops, the heaters switch off after an idle timeout, and temperatures are restored automatically when the program resumes.

All communication uses the fieldbus I/O image (\$IN / \$OUT 1999–4287), declared as KRL SIGNALs in `BasSignals.dat`.

1.1 Relation of other documentation

The software has been developed principally wreferencing the documentation provided with the Dyze controller:

- The Draft version of the RoboSync Hub - User Guide,
- RoboSync Hub - Gateway Mapping

The stepper motor drive has been identified as an an Applied Motion STF06-R DC Advance Microstep Drive. Although not a primary source of information, the applied motion manual provide additional information which can be used to confirm and augment the Dyze information manuals. The following documents are available:

- Quick Setup Guide (920-0140B_STF-R-C-D-IP_Quick_Setup_Guide_RevB.pdf)
- Host-Command-Reference (Host-Command-Reference_920-0002W_0.pdf)
- Electronic Datasheet file: STF06-IP-r3.03.eds
- 2D Drawings: STF03_06-R_RevA.pdf

1.2 Files

The following files constitute the software module

File	Purpose
<code>BasSignals.dat</code>	Signal map for the four PID controllers and the stepper drive. Also declares the <code>Offset</code> and <code>Workpiece</code> frames, which are currently unused.
<code>RoboSync/RoboSyncPidControl.src/.dat</code>	Temperature, PID gain, autotune and alarm



File	Purpose
	functions, plus the PID channel enum, autotune constants and default limits.
RoboSync/RoboSyncStepper.src/.dat	Stepper drive commands and parameters, plus the drive opcode constants and the parameter enum.
RoboSync/RoboSyncSps.src/.dat	Background logic called from <code>sps.sub</code> : error stop, the stepper state machine, the heater idle timeout and program-state handling. Also holds the global flags and the stepper timeout.
RoboSync/RoboSyncSpeed.src/.dat	Converts robot TCP speed into the extruder jog speed. Also holds the material and bead parameters.
RoboSync/RoboSyncInterrupt.src	Interrupt routine that runs when a stopped program resumes.
RoboSync/RoboSyncMsgs.src/.dat	Operator messages for stepper and PID controller alarms (section 8.6). Called from its own submit interpreter, separate from the control SPS.
RoboSync/RoboSyncPrint.src/.dat	Print start and finish sequences called from the robot program: <code>StartPrint</code> and <code>FinishPrint</code> (see Starting and finishing a print).

1.3 Architecture

Layer	Runs in	Responsibility
Robot program (your code)	Robot interpreter	Sets temperatures, enables the stepper, and switches <code>bExtrude</code> on and off along the path.
RoboSyncPidControl, RoboSyncStepper	Robot interpreter; the routines without <code>WAIT</code> are also called by the SPS	Convert engineering units to and from the fieldbus words.
RoboSyncSps	Submit interpreter (<code>sps.sub</code>), every cycle	Drives the stepper state machine, updates the jog speed from the robot speed, stops the



Layer	Runs in	Responsibility
		program on errors, and runs the heater timeout.
RoboSyncInterrupt	Robot interpreter, as an interrupt	Recovery on program resume.
SST gateway and RoboSync Hub	Hardware	PID controllers 0–3 and the stepper drive.

2 Function reference

2.1 RoboSyncPidControl

Every function takes `fi_pidType` ([ENUM_PID_TYPE](#)) to select the channel. For an unknown channel, getters return 0 / 0.0 / FALSE and setters do nothing. Getters marked *commanded* read back the robot's own output word, not the value in the controller.

Routine	Returns	Description
<code>SetTemperature(pid, temp)</code>	–	Sets the setpoint (°C). Recommended way to set a temperature. If the setpoint is outside the current limits, the limits are widened first: low limit = temp × pidDefaultLowerLimit (0.75), high limit = temp × pidDefaultUpperLimit (1.25). Limits are only ever widened.
<code>PidSetPoint(pid, temp)</code>	–	Writes the setpoint directly, leaving the limits unchanged. The controller rejects a value outside SPLL...SPHL.
<code>GetSetTemperature(pid)</code>	REAL °C	Commanded setpoint
<code>GetTemperature(pid)</code>	REAL °C	Measured temperature
<code>SetLowerTemperature(pid, temp) / SetUpperTemperature(pid,</code>	–	Write SPLL / SPHL (°C)



Routine	Returns	Description
temp)		
GetLowerTemperature(pid) / GetUpperTemperature(pid)	REAL °C	Commanded SPLL / SPHL
SetAlarmLimit(pid, temp) / GetAlarmLimit(pid)	– / REAL °C	Write the alarm high limit / read the limit the controller reports
CheckAtTemperature(pid, tol)	BOOL	TRUE if the measured temperature is within $\pm\text{tol}$ °C of the setpoint (strict comparison)
WaitForExtruderTemps()	–	Waits until EXTRUDER0 and EXTRUDER1 are both within rTemperatureTolerance (10 °C) of their setpoints. No timeout. The hub guide recommends ± 2 °C held for 30 s before printing.
SetExtruderHeatZones(t1, t2)	–	Calls SetTemperature for EXTRUDER0 (top, t1) and EXTRUDER1 (bottom, t2)
PidError(pid)	BOOL	TRUE if any of alarm 1–4 is set
GetPidKp / GetPidKi / GetPidKd(pid)	INT	Gains reported by the controller. Only valid while autotune is not running.
SetPidKp / SetPidKi / SetPidKd(pid, val)	–	Write gains through the pidN_SetKx outputs. Autotune must not be running.
SetAutotune(pid, type)	–	Sends an autotune command: AUTOTUNE_STOP, _FAST or _FULL
GetAutotuneBit(pid)	BOOL	TRUE while autotune is running
AutoTuneProcess(pid, temp, type)	–	Stops any autotune in progress, sets the target temperature, starts the autotune and waits for it to finish. Returns without waiting if the controller doesn't



Routine	Returns	Description
		report the start within <code>nAutotuneStartTimeout</code> (5000 ms). The wait for completion has no timeout.

Autotune procedure (hub guide): switch all other heaters off, then tune one at a time in the order bed → top → bottom → nozzle, leaving each tuned heater on while tuning the next.

2.2 RoboSyncAStepper

Routine	Returns	Description
<code>SetStepperParam(param, val)</code>	–	Writes one drive parameter (<code>ENUM_STEPPER_PARAM</code>) in the units of section 3. <code>#JOG_SPEED</code> accepts negative values. It writes the outputs directly, without taking control from the SPS, so don't use it for a parameter the SPS is writing at the same time (for example the jog speed while RUNNING).
<code>StepperOpcode(val)</code>	–	Writes a command to the opcode register. The drive acts on a change of value, so don't write two opcodes back to back without waiting for the drive to respond.
<code>StartJog(speed)</code>	–	Manual jog for purging or loading filament. Resets alarms, enables the drive, sets the jog speed (0.25 RPM units) and starts jogging, waiting 0.2 s between each step. Jog accel/decel are left at the drive's power-up value. Robot interpreter only. Don't use while the SPS controls the stepper.
<code>StopJog()</code>	–	Stops manual jogging
<code>StepperError()</code>	BOOL	TRUE if any alarm bit is set



Routine	Returns	Description
ToUnsigned16(val)	INT	Encodes a signed value as 16-bit two's complement for an output word (−100 → 65436)
Truncate16(n)	INT	Keeps the low 16 bits. Unused and untested.
RoboSyncSpeed(speed)	INT	Converts a TCP speed (m/s) into the jog-speed register value (see below), rounded to an INT.

2.3 RoboSyncPrint

Routine	Description
StartPrint()	Prepares to print. Sets rFilamentOD (2.85 mm), rLayerHeight and rLayerWidth (0.5 mm), and rExtrusionConstant (1.0); sets both extruder heat zones to 180 °C with SetExtruderHeatZones and waits for them with WaitForExtruderTemps; then sets bSpsControl TRUE, bEnableRetract and bExtrude FALSE, bEnableStepper TRUE, and waits until the stepper is STOPPED (drive enabled and ready) or ERROR. Robot interpreter only.
FinishPrint()	Finishes a print. Sets bExtrude, bEnableRetract and bEnableStepper FALSE, waits until the stepper is DISABLED or ERROR, then sets both extruder heat zones to 0 °C. It doesn't wait for the heaters to cool. Robot interpreter only.

2.3.1 Extrusion speed calculation

```

VolPerSec = rLayerHeight x rLayerWidth x (speed x 1000) [mm3/s]
VolPerRev = (PI x rFilamentOD^2 / 4) / (rStepsPerMM / rStepsPerRev)
[mm3/rev]
JogSpeed = rExtrusionConstant x 240 x VolPerSec / VolPerRev

```

240 = 60 (rev/s to RPM) × 4 (RPM to the 0.25 RPM register unit).

The result is clamped to 0–19200 and rounded to an INT. With the defaults (2.85 mm filament, 274 steps/mm, 3200 steps/rev), one motor revolution extrudes 74.5 mm³, matching the hub guide. Use



`rExtrusionConstant` to calibrate the flow: for example, if a wall measures 2.20 mm instead of 2.40 mm, multiply it by $2.40 / 2.20$.

TCP speed (0.5 × 0.5 mm bead)	10 mm/s	20 mm/s	50 mm/s	100 mm/s	200 mm/s
Motor speed	2.0 RPM	4.0 RPM	10.1 RPM	20.1 RPM	40.3 RPM
Jog register	8	16	40	81	161

3 Background Logic – RoboSyncSps

3.1 Runs Per cycle

- **Boot detection.** If `oStartUpFlag` is FALSE (the controller has just booted), `stepperState` is set to #DISABLED and the flag is set.
- **Error stop.** `bStepperError = StepperError()` OR `bTimeoutError` and `bPidError = PidError(#EXTRUDER0) OR PidError(#EXTRUDER1)`. While either is TRUE, STOP 1 is sent **every cycle**, so the program can't be continued until the error clears.
- **Stepper state machine** (8.2), only while `bSpsControl` is TRUE.
- **Program-state handling** (8.3).
- **Heater idle timeout** (8.4).

3.2 Stepper State Machine

The state machine waits on the drive's status bits (section 4.4). One timeout, `nStepperTimeout` (2000 ms), guards each state that waits on the drive. `StartStepperTimer` restarts it and clears `bTimeoutError`. When it expires, the SPS sends STOP_JOGGING, sets `bTimeoutError` and goes to ERROR. Short helper steps are written inline in each state, to make debugging on the robot easier. Before the state logic runs, every cycle, if `bStepperError` is TRUE in any state except DISABLED, RESET, RESETTING or ERROR, the SPS sends STOP_JOGGING and moves to ERROR. Those four are excluded so that a reset can run: RESET sends the alarm reset and clears `bTimeoutError`.

State	Action	Next state
DISABLED	Drive disabled. None	RESET, once <code>bEnableStepper</code> is TRUE
RESET	Clears <code>bResetStepper</code> , sends RESET_ALARM_CODE, starts the timeout	RESETTING
RESETTING	When <code>stepper_fault</code> is FALSE: sends ENABLE_DRIVE	ENABLED; ERROR on timeout



State	Action	Next state
	and restarts the timeout	
ENABLED	Waits for <code>stepper_enabled</code>	STOPPED; ERROR on timeout
STOPPED	Drive enabled and idle. When <code>bEnableStepper</code> and <code>bExtrude</code> are TRUE and the stepper isn't overridden: writes jog speed 0, sends <code>COMMENCE_JOGGING</code> and starts the timeout. When <code>bEnableStepper</code> is FALSE: sends <code>DISABLE_DRIVE</code> and starts the timeout	STARTING; DISABLING when the stepper is disabled
DISABLING	Waits for <code>stepper_enabled</code> to go FALSE	DISABLED; ERROR on timeout
STARTING	Waits for <code>stepper_jogging</code> . If extrusion is switched off first, sends <code>STOP_JOGGING</code>	RUNNING; STOPPING if extrusion is switched off; ERROR on timeout
RUNNING	While extruding, writes the jog speed each time it changes: <code>nCommandedFeedSpeed</code> when <code>bCommandedFeed</code> is TRUE, otherwise calculated from <code>\$VEL_ACT</code> (or the programmed <code>\$VEL.CP</code> when the robot is stationary, so purging in place works).	RETRACT, if extrusion stops and <code>bEnableRetract</code> is set; STOPPING (after <code>STOP_JOGGING</code>), if extrusion stops without retract or <code>bEnableStepper</code> is FALSE
RETRACT	Keeps jogging, in reverse at <code>-nRetractSpeed</code> , for <code>nRetractTime</code> ms, then sends <code>STOP_JOGGING</code> and starts the timeout	STOPPING
STOPPING	Waits until the drive reports neither <code>stepper_moving</code> nor <code>stepper_stopping</code> . Times out on the stepper timer started on entry.	STOPPED; ERROR on timeout
ERROR	Stops the timer. Sends <code>DISABLE_DRIVE</code> while the motor is not moving and	RESET, when <code>bResetStepper</code> is TRUE



State	Action	Next state
	stepper_enabled is still TRUE	

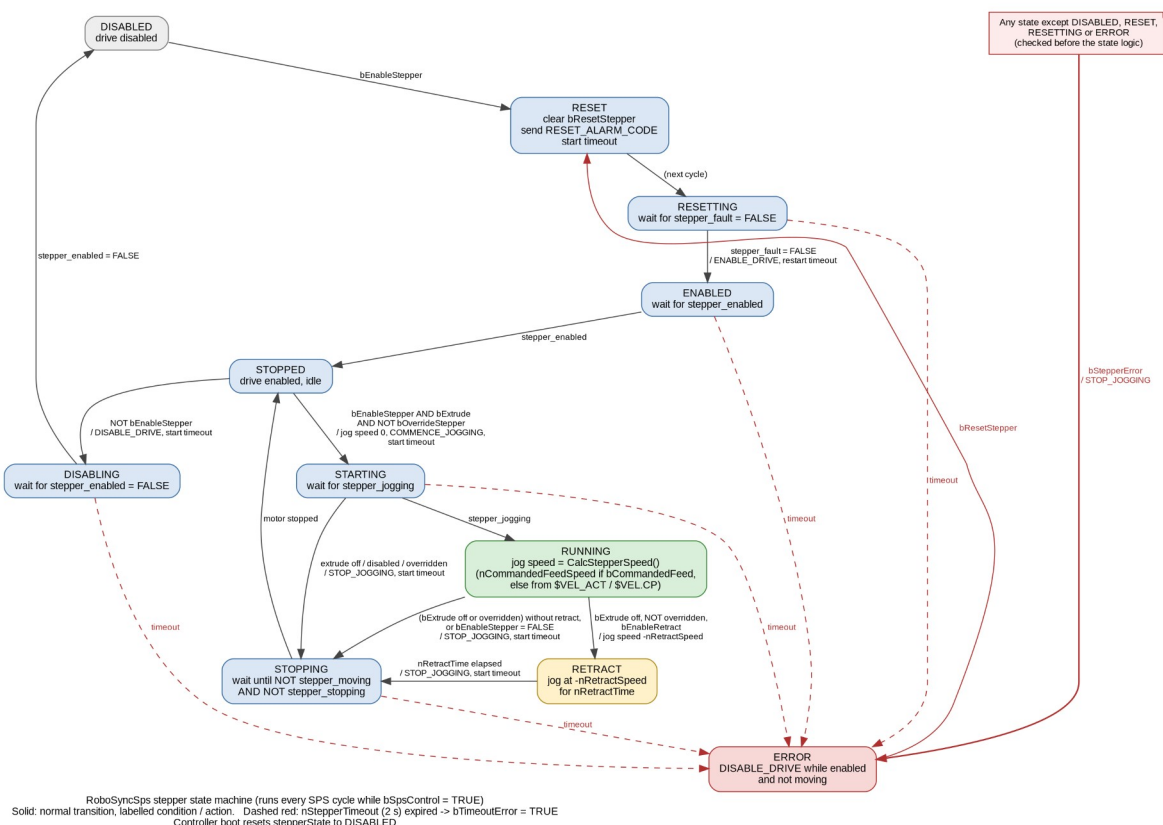


Figure 1: Stepper Control State Diagram

Extrusion starts as soon as the drive reports `stepper_jogging`: the motor starts at speed 0 and `RUNNING` writes the real speed on its first cycle. The SPS does not write the jog acceleration and deceleration; unless otherwise set the drive uses its power-up value (1000 rps/s).

A heater temperature alarm or an E-stop inside the hub disables the stepper drive. The drive re-enables itself afterwards but won't turn until it receives a new movement command. When the robot program is stopped as well (by an extruder alarm, which the SPS detects, or by an E-stop that stops the robot), the stepper is stopped with the program and the resume goes through `STOPPED` → `STARTING`, which sends that command. A bed or nozzle alarm does **not** stop the program (see section 13).

Program state (`$PRO_STATE1`)

Program state	Effect
#P_FREE (no program selected) or #P_RESET	Stepper overridden (stopped). No resume



Program state	Effect
	interrupt pending. Saved temperatures discarded, so a newly started program sets its own. The heater idle timer runs.
#P_STOP (program stopped)	Stepper overridden, so extrusion stops with the program. The resume interrupt is armed (<code>bRoboSyncInterrupt</code>). The heater idle timer runs.
#P_ACTIVE (program running)	Stepper override cleared. The heater idle timer is stopped and reset.
#E-stop (<code>\$ALARM_STOP FALSE</code>)	No separate handling: the controller stops the program, so the #P_STOP behaviour applies, including the heater idle timer.

3.2.1 Heater Idle timeout

The heater idle timer runs whenever no program is active: no program selected (`#P_FREE`), program stopped (`#P_STOP`), including after an E-stop, or reset. It is stopped and reset while the program runs (`#P_ACTIVE`).

When the timer reaches `nHeaterTimeout` (600000 ms, 10 minutes), the SPS saves the four setpoints in `SaveTemperature[1..4]` (bed, top, bottom, nozzle), sets `bTemperatureSave`, and sets all four setpoints to 0, so an unattended system does not sit at temperature. The heaters then stay off until the setpoints are set again: the resume interrupt restores the saved setpoints when the stopped program is restarted, and if the program is reset or deselected instead, the saved setpoints are discarded and the program must set new ones. Heaters are not switched off immediately on an E-stop by software, by design: because of the thermal mass, it doesn't increase risk, and it allows a quick recovery. Whether the hub cuts heater power on an E-stop depends on how its 24 V Safety input is wired.

3.2.2 Resume interrupt `RoboSyncInterrupt`

The interrupt fires when a stopped program is restarted. It does the following:

1. Brakes the robot.
2. If the heaters were switched off by the timeout (`bTemperatureSave`), restores the saved setpoints and clears the flag.
3. Waits for both extruder zones to reach temperature (`WaitForExtruderTemps`, no timeout).
4. If the stepper has an alarm, requests a reset.
5. Waits for the stepper to be `RUNNING`; or `STOPPED` if `bExtrude` is off; or `DISABLED` if the stepper is disabled.
6. Clears `bRoboSyncInterrupt`, and the program continues.



3.3 Operator Messages

`RoboSyncMsgs()` runs in its own submit interpreter (section 2), not in `RoboSyncSps`, so message handling can't interfere with the control logic. For each alarm bit it calls `CheckWarningMsg`, which shows a notify message on the smartPAD once when the bit goes TRUE and re-arms it when the bit clears. The texts are in `RoboSyncMsg[1..58]` in `RoboSyncMsgs.dat`. The message number shown is `1 + nNo`.

Messages	Source
1–14	Stepper alarm code bits (position limits, temperature, voltage, current, winding, encoder, comms, flash, no move, blank Q segment)
15–25	Heated bed, PID 0: alarms 1–4 and status word 3 conversion errors
26–36	Top heater (Extruder0), PID 1: as above
37–47	Bottom heater (Extruder1), PID 2: as above
48–58	Nozzle, PID 3: as above

Other routines in the module: `WarningMsg` (show once), `DisplayMsg` (show every call) and `WarningClear` (re-arm).



3.4 Configuration Variables

Variable	Default	Module	Meaning
bSpsControl	FALSE	SPS	Lets the SPS drive the stepper. You must set this to TRUE.
bEnableStepper	FALSE	SPS	Enables the extruder stepper
bExtrude	FALSE	SPS	Extrude now. Switch on and off along the path
bCommandedFeed / nCommandedFeedSpeed	FALSE / 0	SPS	When bCommandedFeed is TRUE, RUNNING uses nCommandedFeedSpeed (jog register units, 0.25 RPM per count) set by the program, for example from CAD/CAM, instead of calculating the speed from the robot. It is not scaled by rExtrusionConstant or clamped.
bEnableRetract	FALSE	SPS	Retract when extrusion stops
nRetractSpeed / nRetractTime	100 / 500 ms	SPS	Retract speed (0.25 RPM units, so 100 = 25 RPM) and duration
nStepperTimeout	2000 ms	SPS	Time allowed for the drive to reset, enable, start or stop before the state machine goes to ERROR
bTimeoutError	FALSE	SPS	Set when nStepperTimeout expires; stops the program until the stepper is reset (read only)



Variable	Default	Module	Meaning
bResetStepper	FALSE	SPS	Set to TRUE to reset the stepper from ERROR
nHeaterTimeout	600000 ms	SPS	Idle time before the heaters switch off
stepperState	#DISABLED	SPS	Current state of the stepper state machine (read only)
bPidError / bStepperError	FALSE	SPS	Error status (read only)
rTemperatureTolerance	10 °C	PidControl	Tolerance for WaitForExtruderTemps
pidDefaultLowerLimit / pidDefaultUpperLimit	0.75 / 1.25	PidControl	Factors SetTemperature uses to widen the limits
nAutotuneStartTimeout	5000 ms	PidControl	How long AutoTuneProcess waits for autotune to start
rFilamentOD	2.85 mm	Speed	Filament diameter
rLayerHeight / rLayerWidth	0.5 / 0.5 mm	Speed	Bead cross-section
rStepsPerMM / rStepsPerRev	274 / 3200	Speed	Feed mechanics (hub guide values)
rExtrusionConstant	1.0	Speed	Flow calibration factor



4 Using RoboSync module In a program

4.1 Example program

```
DEF ExamplePrint()  
;FOLD INI  
;FOLD BASISTECH INI  
GLOBAL INTERRUPT DECL 3 WHEN $STOPMESS==TRUE DO IR_STOPM ( )  
INTERRUPT ON 3  
GLOBAL INTERRUPT DECL 4 WHEN bRoboSyncInterrupt DO RoboSyncInterrupt()  
INTERRUPT ON 4  
BAS (#INITMOV,1 )  
;ENDFOLD (BASISTECH INI)  
  
;FOLD USER INI  
;Make your modifications here  
  
;ENDFOLD (USER INI)  
;ENDFOLD (INI)  
  
HomePosition()  
PrintSafe()  
  
StartPrint()  
  
PTP P_Start  
  
; Extrude along the path, switching bExtrude on the path itself  
TRIGGER WHEN DISTANCE=0 DELAY=0 DO bExtrude=TRUE  
LIN P1  
LIN P2  
TRIGGER WHEN DISTANCE=1 DELAY=0 DO bExtrude=FALSE  
LIN P3  
...  
  
PrintSafe()  
HomePosition ()  
  
END
```

4.2 Switching extrusion on the path

In KRL, a plain assignment such as `bExtrude = TRUE` runs in the **advance run**, ahead of the robot's actual position. Use `TRIGGER WHEN DISTANCE ... DO bExtrude = ...` (as above), or stop the advance run first, so extrusion starts and stops where it should. Extrusion starts once the drive reports it is jogging, so allow a short start-up delay, for example with the TRIGGER's `DELAY`.

4.3 Starting and finishing a print

`StartPrint()` (module `RoboSyncPrint`) prepares the extruder. It sets `rFilamentOD`, `rLayerHeight`, `rLayerWidth` and `rExtrusionConstant`, sets both extruder heat zones to 180 °C and waits for them to reach temperature. It then enables the stepper under SPS control, with



extrusion and retract off, and returns when the drive is ready (STOPPED), or in ERROR. Edit the values in `StartPrint` to suit the material and bead.

`FinishPrint()` switches extrusion off, disables the stepper and waits for DISABLED (or ERROR), then sets both extruder heat zones to 0 °C. It doesn't wait for the heaters to cool.

4.4 Commanded feed

By default the extruder speed is calculated from the robot's TCP speed. When the program supplies the feed itself, for example from a CAD/CAM post-processor, set `bCommandedFeed` to TRUE and write the speed to `nCommandedFeedSpeed` in jog register units (0.25 RPM per count, so the value is $4 \times \text{RPM}$). RUNNING then uses that value and updates the drive each time it changes. It isn't scaled by `rExtrusionConstant` or clamped, so keep it within ± 19200 . The `bExtrude` can still be used to turn on and off the extrusion, particularly if the reverse movement is required, alternatively the post processor could be set up to set the `nCommandedFeedSpeed` to 0

```
bCommandedFeed = TRUE
TRIGGER WHEN DISTANCE=0 DELAY=0 DO nCommandedFeedSpeed=80      ; 20 RPM
TRIGGER WHEN DISTANCE=0 DELAY=0 DO bExtrude=TRUE
LIN P1
TRIGGER WHEN DISTANCE=0 DELAY=0 DO nCommandedFeedSpeed=120     ; 30 RPM
LIN P2
TRIGGER WHEN DISTANCE=1 DELAY=0 DO bExtrude=FALSE
LIN P3
bCommandedFeed = FALSE
```

4.5 Purging in place

With the robot stationary, RUNNING uses the programmed CP speed `$VEL.CP`, so the purge rate follows it. With commanded feed, set `nCommandedFeedSpeed` instead.

```
WAIT SEC 0                ; stop the advance run
$VEL.CP = 0.05             ; purge rate follows the programmed CP speed
bExtrude = TRUE
WAIT SEC 5
bExtrude = FALSE
```

4.6 Autotuning

Switch all other heaters off, then tune one at a time in the order bed, top, bottom, nozzle, leaving each tuned heater on while tuning the next.

```
AutoTuneProcess(#HEATED_BED, 60.0, AUTOTUNE_FAST)
AutoTuneProcess(#EXTRUDER0, 220.0, AUTOTUNE_FAST)
AutoTuneProcess(#EXTRUDER1, 220.0, AUTOTUNE_FAST)
```

4.7 Recovering from a stepper error

While a stepper alarm or `bTimeoutError` is active, the SPS sends `STOP 1` every cycle, so the program itself can't run the recovery. Clear the cause, then set `bResetStepper` from outside the program, for example from an HMI button or the variable display. The state machine goes through



RESET, which sends the alarm reset and clears `bTimeoutError`, and re-enables the drive (STOPPED).

```
; e.g. from an HMI routine
IF stepperState == #ERROR THEN
    bResetStepper = TRUE
ENDIF
```

5 Commissioning checklist

1. Build the project in WorkVisual or OfficeLite with no errors. Make sure the old `RoboSyncControl` module is gone.
2. Check the gateway mapping in WorkVisual against the I/O map.
3. Confirm that timers 16–19 and `$OUT[1999]` are free, and that your KSS version allows `$TIMER_FLAG[18]` to be written (RoboSyncSps does this).
4. Check how the gateway handles the robot's output words at boot, when they are all 0. If they are passed straight to the hub, the gains, alarm limits and SPHL would be written as 0.
5. For each fitted heater, set a temperature and confirm that the correct controller responds and that `GetTemperature` reads it back. Check that `pidN_Run` is TRUE while a controller is running.
6. Write and read back Kp, Ki and Kd on each channel. Run a fast autotune in the order bed, top, bottom, nozzle.
7. With the drive powered, check that `stepper_enabled`, `stepper_fault`, `stepper_jogging`, `stepper_moving` and `stepper_stopping` follow the drive. If they stay FALSE, every transition will time out into ERROR.
8. Set `bSpsControl` and `bEnableStepper` and follow `stepperState` through DISABLED → RESET → RESETTING → ENABLED → STOPPED. Set `bExtrude` and check STARTING → RUNNING with the motor turning. Clear `bEnableStepper` and check STOPPING → STOPPED → DISABLING → DISABLED.
9. Unplug the motor: `stepper_alarm` should read 256, message 9 should appear if the message submit interpreter is running, the program should stop, and the state machine should go to ERROR and disable the drive. Reconnect, set `bResetStepper` and check that it recovers to STOPPED.
10. Test a retract with `bEnableRetract TRUE`: switching `bExtrude` off should go RUNNING → RETRACT (motor reverses for `nRetractTime`) → STOPPING → STOPPED, and the next extrusion should run forwards.
11. Stop the program while extruding: the extruder must stop. Restart it: the interrupt must complete with extrusion on, with it off, and with the stepper disabled.
12. Leave the program stopped for longer than `nHeaterTimeout`: all heaters must switch off. Resume: the temperatures must be restored. Repeat once to confirm the timeout works a second time.
13. Set `bCommandedFeed` and a value in `nCommandedFeedSpeed`, and check the motor runs at the value ÷ 4 RPM.
14. Run `StartPrint` and `FinishPrint` end to end.
15. Calibrate `rExtrusionConstant` against a measured wall, starting from 1.0.



B Automation Services

RoboSync Hub Custom KRL Software

